

Drone Security Analyst Agent

Code Documentation

1. Framework and Architecture

The codebase is organised into four packages, each a clear layer with a single responsibility, plus a database schema and a container definition. Every layer below blob storage is source agnostic, so the simulated feed can be replaced by a real stream without touching the agent, backend, or frontend.

Package layout

data/ frame production: video sampling, blob storage, ingest runner
agent/ perception + reasoning: graph, nodes, rules, llm, db, embedder
 nodes/ the seven LangGraph nodes
backend/ FastAPI app + REST routes
frontend/ React + Tailwind dashboard
db/init.sql frames / events / alerts schema with pgvector
tests/ unit, agent, API, and QA scenario suites

The agent framework is LangGraph. One HTTP call to the ingest endpoint runs a graph of seven nodes with conditional edges. A typed state object is threaded through the nodes, and a process level session store provides memory across frames. The language model is bounded to perception and narrative, while a deterministic rule engine owns the decision of whether an alert fires.

2. Technology Stack

Pinned versions from requirements.txt, with the role each component plays.

Component	Version	Role
fastapi / uvicorn	0.115 / 0.34	Async REST API and ASGI server
langgraph	0.2.62	Stateful agent graph with conditional routing
langchain-openai	0.3.0	OpenAI compatible client wiring
openai	1.59	Chat, vision, and embeddings client (Grok + OpenAI)
sqlalchemy[asyncio]	2.0.36	Async ORM core
asyncpg	0.30	PostgreSQL async driver
pgvector	0.3.6	Vector column type and similarity search
boto3	1.35	S3 API client for MinIO
opencv-python-headless	4.10	Frame sampling from video clips
pillow	11.1	JPEG encoding of sampled frames
pydantic	2.10	Request validation and typed models
pytest / pytest-asyncio	8.3 / 0.25	Test runner, async tests
httpx	0.28	HTTP client for the live API and QA tests

Models in use

Task	Provider	Model
Frame vision	Groq	meta-llama/llama-4-scout-17b-16e-instruct
Reasoning narrative	Groq	llama-3.3-70b-versatile
Chatbot reasoning loop	OpenAI	gpt-4o-mini
Embeddings	OpenAI	text-embedding-3-small (local hashing fallback)

3. Module Reference

data/ : frame production

- **video_loader.sample_frames(clip, interval_seconds, max_frames, seed)** opens a clip with OpenCV, samples one frame every N seconds of video time, JPEG encodes it, and yields (frame_bytes, telemetry). Telemetry is seeded so a clip plus interval plus seed reproduces identical output.
- **blob_store.upload_frame(image_bytes, filename)** puts bytes into MinIO and returns a browser reachable URL; **get_frame_bytes(blob_url)** fetches them back. Two endpoints are used by design: an internal one for byte access and a public one baked into stored URLs for the browser.
- **ingest_runner.run(...)** drives the whole pipeline: sample frames across clips, upload each, POST to the ingest endpoint, and print a per frame line plus an aggregate of routes, events, and alerts.

agent/ : perception and reasoning

- **graph.run_frame(blob_url, telemetry)** compiles the graph once (cached) and runs one full cycle, returning the API result payload.
- **nodes/analyze_frame.analyze_frame(blob_url)** fetches the image, calls the vision model, and validates the response into a fixed schema.
- **nodes/query_history.node** embeds the description and runs a pgvector similarity search and a SQL recency filter in parallel.
- **rules.evaluate(vlm, telemetry, rolling_window, events_today)** returns the list of fired rules; pure and unit testable, no API call.
- **llm.complete_json(...)** returns a parsed JSON object from the first working provider; **complete_with_tools(...)** supports the chatbot tool calling loop. Both accept an explicit provider argument.
- **embedder.embed_text(text)** returns a 1536 dimension vector from OpenAI or a deterministic local hashing fallback.
- **db** holds async helpers: `insert_frame`, `insert_event`, `insert_alert`, `query_similar_frames`, `query_recent_by_location`, `fetch_events`, `fetch_alerts`, `fetch_recent_frames`, `fetch_summary_stats`.
- **state** defines the AgentState type and the SessionStore that carries the rolling window and this session's events and alerts across frames.

backend/ : API surface

Route	Method	Purpose
/ingest	POST	Run the full agent on a blob_url plus telemetry
/ingest_image	POST	Multipart image upload, store, then run the agent
/frames	GET	Recent frames (drives the live telemetry feed)
/frames/search	GET	pgvector semantic search, returns blob_url per match
/events	GET	Event log
/alerts	GET	Alerts, with active_only filter
/summary	GET	Aggregate stats plus an LLM session summary
/chat	POST	Grounded Q and A with a reasoning loop and vision tool
/health	GET	Liveness

4. Key Implementation Details

Conditional routing in the graph

reason_decide emits a route, and two conditional edge maps fan the flow out. A normal frame goes straight to persistence; log and alert pass through log_event first; only alert continues to trigger_alert.

```
g.add_conditional_edges("reason_decide", _route,
    {"normal": "update_state", "log": "log_event", "alert": "log_event"})
g.add_conditional_edges("log_event", _route,
    {"alert": "trigger_alert", "log": "update_state", "normal": "update_state"})
```

Deterministic rules separate from the model

Alert firing is decided by a pure function so behaviour is reproducible and testable. The model supplies narrative only and cannot suppress a fired rule.

```
fired = rules.evaluate(vlm, telemetry, rolling_window, events_today)
if fired:
    route = 'alert'
elif noteworthy:
    route = 'log'
else:
    route = 'normal'
```

Provider split with a single client abstraction

Groq and OpenAI both expose an OpenAI compatible chat API, so one client serves both. Each call passes an explicit provider, so vision and reasoning stay on Groq while the chatbot reasoning stays on the OpenAI mini model, regardless of global toggles.

Chatbot reasoning loop with an on demand vision tool

The chat route is a short tool calling loop rather than a single shot. It retrieves frames, reasons over their metadata, and may call the vision model on a specific frame when the text index is insufficient, then folds the visual findings into the answer.

5. Validation of Functionality

Validation runs at three levels: unit and agent tests close to the code, a live API suite over the running service, and a scenario harness that drives real video sourced frames through the system and checks operator facing behaviour. All nine automated tests pass against the running stack.

5.1 Unit and agent tests

tests/test_analyze_frame.py samples five real frames into MinIO and asserts the vision step returns the full schema with valid types, plus a valid embedding. The key assertions:

```
for field in (object_type, location, action, clothing, color, confidence):
    assert field in result and result[field] not in (None, '')
assert 0.0 <= result['confidence'] <= 1.0
assert len(embed_text(desc)) == 1536
```

tests/test_agent_pipeline.py runs the full graph on six real frames via `run_frame` and asserts both the returned result and the database writes:

```
assert result['status'] == 'ok'
assert result['decision']['route'] in {normal, log, alert}
# then, reading back the frames table:
assert row['blob_url'].startswith('http')
assert row['has_emb'] and row['dims'] == 1536
```

tests/test_api.py exercises every REST endpoint against the live backend and asserts, as the central requirement, that frame URLs are present and resolve to real images served by MinIO.

5.2 Scenario harness for dynamic inputs

The dynamic input is a real run of sampled video frames with seeded synthetic telemetry. Twenty five frames were driven through the live agent. Because the synthetic time of day is spread across a full 24 hour cycle, one run produces both routine daytime frames and after hours frames, which exercises the emergency path without any staged input. The run produced 22 alert routes, 2 normal, and 1 log, with after hours vehicle firing 10 times, after hours person 8 times, and repeat vehicle 8 times across 30 frames in the database.

The harness, `tests/qa_scenarios.py`, queries the live endpoints and reports each case. Result: 7 pass, 2 informational, 0 fail.

Case	Scenario	Outcome
QA-01	Vehicle frame persisted with URL	Pass, 20 vehicle frames with URLs
QA-02	Repeat or after hours vehicle alert	Pass, 18 vehicle alerts
QA-03	After hours pedestrian alert	Pass, 8 alerts
QA-04	Loitering across 5 frames	Info, rule verified directly
QA-05	Frame search returns images	Pass, rows carry blob_url
QA-06	Search image is fetchable	Pass, HTTP 200 image/jpeg
QA-07	Chat answer grounded in frames	Pass, answer cites a frame
QA-08	Session summary generated	Pass, over 30 frames and 26 alerts
QA-09	Normal daytime activity	Info, daytime alerts are repeat_vehicle

5.3 Emergency response scenarios

The emergency path is the route from a rule violation to a persisted alert that carries the triggering frame. These were observed live during the run:

- **After hours person (critical).** A person detected outside business hours fires `after_hours_person`. Example message: `person at loading_dock, 04:48, after hours`.
- **After hours vehicle (high).** A vehicle outside business hours fires `after_hours_vehicle`; ten such alerts were raised in the run.
- **Repeat vehicle (warning).** The same vehicle descriptor seen three or more times in a session fires `repeat_vehicle`; eight were raised, and this rule is intentionally not time gated.
- **Loitering (high).** Five consecutive frames of the same subject at one location fire `loitering`. It does not arise from the ingest run because telemetry cycles the waypoint each frame, so it was verified by direct evaluation, which produced the alert as expected.
- **Restricted zone (critical).** Any subject in a configured restricted location fires `restricted_zone` when that configuration is set.

In every alerting case the alert row stores the rule, message, severity, and the triggering frame URL, which is what the dashboard alert banner renders. If the reasoning model call fails, the node falls back to a deterministic summary, so an alert is never lost to a model error.

5.4 Running the tests

```
# unit and agent tests
pytest tests/test_analyze_frame.py tests/test_agent_pipeline.py
```

```
# live API tests (stack must be up)
pytest tests/test_api.py
```

```
# populate data, then run the scenario harness
python -m data.ingest_runner --frames 25 --interval 2.0
python tests/qa_scenarios.py
```

Tests that need a model key or a reachable service skip themselves cleanly when those are absent, so the suite stays green in restricted environments rather than reporting false failures.